

Fundamentos de Concurrency en Go: El Modelo GMP y su Implementación Técnica

1. Introducción: Concurrency vs. Paralelismo

En la ingeniería de software moderna es fundamental distinguir los conceptos de **concurrency** y **parallelismo**, ya que suelen utilizarse indistintamente aunque representan ideas diferentes.

La **concurrency** es un modelo de diseño de software que permite estructurar múltiples tareas independientes para progresar de manera intercalada. No implica necesariamente ejecución simultánea, sino la capacidad del sistema de administrar varias tareas activas al mismo tiempo.

El **parallelismo**, en cambio, es una característica del hardware que permite ejecutar múltiples tareas físicamente al mismo tiempo utilizando varios núcleos de procesamiento.

Un programa concurrente puede ejecutarse en un único núcleo alternando tareas rápidamente, mientras que un programa paralelo requiere múltiples núcleos ejecutando instrucciones simultáneamente.

El lenguaje Go fue diseñado específicamente para simplificar el desarrollo de software concurrente. Para ello, implementa un Runtime propio que abstrae la complejidad de la administración manual de hilos del sistema operativo (OS Threads), permitiendo que el desarrollador trabaje con unidades livianas llamadas **Goroutines**.

2. La Arquitectura del Scheduler: El Modelo GMP

Go implementa un scheduler de tipo **M:N**, donde múltiples Goroutines son multiplexadas sobre un conjunto reducido de hilos reales del sistema operativo.

El Runtime organiza este mecanismo utilizando el modelo **GMP**, compuesto por tres entidades fundamentales:

G — Goroutine

La Goroutine representa la unidad mínima de ejecución concurrente en Go.

A diferencia de los threads tradicionales del sistema operativo, las Goroutines son extremadamente livianas:

- Un thread del sistema suele reservar aproximadamente **1 MB** de stack.
- Una Goroutine comienza con un stack dinámico cercano a **2 KB**.

El stack de una Goroutine puede crecer y reducirse dinámicamente según sea necesario, permitiendo la creación de cientos de miles o incluso millones de Goroutines dentro de un mismo proceso.

Cuando una función es invocada utilizando la palabra reservada **go**, el Runtime:

1. Crea una nueva estructura **G**.
2. Inicializa su stack.
3. La coloca en una cola de ejecución del scheduler.

Ejemplo:

```
go procesarPedido()
```

M — Machine (OS Thread)

La entidad **M** representa un hilo real del sistema operativo.

Es la estructura encargada de ejecutar instrucciones sobre el CPU. Sin embargo, una **M** no puede ejecutar Goroutines por sí sola: necesita asociarse a un procesador lógico (**P**).

Go reutiliza threads automáticamente mediante un thread pool interno, evitando el costo elevado de crear y destruir threads constantemente.

P — Processor

La entidad **P** representa un contexto lógico de ejecución.

Contiene:

- Recursos necesarios para ejecutar Goroutines.
- Cachés del scheduler.
- Colas locales de ejecución (**Local Run Queues**).

La cantidad de **P** disponibles está controlada por la variable:

GOMAXPROCS

Generalmente coincide con la cantidad de núcleos disponibles en la máquina.

La arquitectura se resume así:

- Una **G** necesita una **M** para ejecutarse.
- Una **M** necesita una **P** para poder ejecutar código Go.

Es decir:

$G \rightarrow \text{se ejecuta sobre} \rightarrow M$

$M \rightarrow \text{necesita} \rightarrow P$

2.1 La función **main** también es una Goroutine

Un detalle importante del Runtime de Go es que incluso la función **main()** se ejecuta dentro de una Goroutine.

Cuando un programa Go inicia:

1. El sistema operativo crea el proceso.
2. El Runtime de Go inicializa el scheduler GMP.
3. Se crea la primera Goroutine, conocida internamente como **main goroutine**.
4. Dentro de ella se ejecuta la función:

`func main()`

Es decir, **main()** no corre “fuera” del modelo concurrente: forma parte del mismo scheduler que administra el resto de las Goroutines.

Conceptualmente:

Runtime inicia → crea Goroutine principal → ejecuta main()

Por este motivo, cuando utilizamos:

```
go tarea()
```

la nueva Goroutine queda ejecutándose concurrentemente respecto de la Goroutine principal (**main goroutine**).

Finalización del Programa y Main Goroutine

Un aspecto importante es que el ciclo de vida del programa depende directamente de la Goroutine principal.

Cuando la función **main()** finaliza:

- el proceso termina,
- el Runtime detiene el scheduler,
- y todas las demás Goroutines son abortadas automáticamente.

Ejemplo:

```
func main() {  
    go tarea()  
}
```

En este caso, es posible que **tarea()** nunca llegue a ejecutarse, porque **main()** termina inmediatamente.

Por eso suelen utilizarse mecanismos de sincronización como:

```
sync.WaitGroup  
channels  
context
```

para evitar que la Goroutine principal finalice antes que las tareas concurrentes.

3. Estrategias de Optimización del Scheduler

El scheduler de Go implementa múltiples mecanismos avanzados para maximizar la utilización del CPU y minimizar tiempos ociosos.

Local Run Queue

Cada **P** mantiene una cola local de Goroutines listas para ejecutarse.

Esto evita contención global entre múltiples CPUs y mejora significativamente la performance en sistemas multicore.

Cuando una Goroutine se crea:

```
go tarea()
```

normalmente se agrega a la cola local del **P** actual.

Global Run Queue

Además de las colas locales, existe una cola global compartida.

El scheduler utiliza esta cola para balancear carga cuando:

- una cola local se llena,
 - un **P** queda sin trabajo,
 - o se requiere redistribución de tareas.
-

Work Stealing

Uno de los mecanismos más importantes del scheduler es el **Work Stealing**.

Cuando un procesador lógico (**P**) agota su cola local de tareas:

1. Busca otro **P** con trabajo pendiente.
2. “Roba” aproximadamente la mitad de sus Goroutines.
3. Continúa ejecutándolas localmente.

Esto permite distribuir carga dinámicamente entre núcleos sin intervención del desarrollador.

Handoff ante System Calls Bloqueantes

Cuando una Goroutine realiza una syscall bloqueante (por ejemplo I/O de disco o red):

```
archivo.Read(buffer)
```

el thread (M) podría quedar bloqueado esperando respuesta del sistema operativo.

Para evitar desperdiciar CPU:

1. La M libera temporalmente su P.
2. Otra M toma ese P.
3. El resto de las Goroutines continúan ejecutándose normalmente.

Este mecanismo se denomina **Handoff** y es clave para la eficiencia del Runtime de Go.

Network Poller

Go incorpora además un mecanismo interno llamado **Network Poller** basado en APIs del sistema operativo como:

- **epoll** (Linux)
- **kqueue** (BSD/macOS)
- **IOCP** (Windows)

Esto permite manejar miles de conexiones concurrentes sin crear un thread por conexión, utilizando un modelo asincrónico eficiente integrado al scheduler.

4. Comunicación y Sincronización

Go adopta la filosofía:

“Do not communicate by sharing memory; share memory by communicating.”

En lugar de depender excesivamente de memoria compartida y locks, Go promueve el uso de canales (**channels**) para sincronizar Goroutines.

Channels (**chan**)

Los canales permiten enviar y recibir datos entre Goroutines de forma segura.

Ejemplo:

```
ch := make(chan int)
```

```
go func() {  
    ch <- 10  
}()
```

```
valor := <-ch
```

Características importantes:

- Son thread-safe.
 - Permiten sincronización implícita.
 - Pueden ser bufferizados o no bufferizados.
-

Mutex

Aunque Go favorece channels, también provee primitivas clásicas de sincronización:

```
sync.Mutex  
sync.RWMutex  
sync.WaitGroup  
sync.Cond  
atomic
```

Estas herramientas son útiles cuando múltiples Goroutines acceden a memoria compartida.

5. Palabras Reservadas Relacionadas con Concurrency

Palabra Reservada	Función Técnica
<code>go</code>	Instancia una nueva Goroutine y la agrega al scheduler.
<code>chan</code>	Define un canal de comunicación concurrente.
<code>select</code>	Permite esperar múltiples operaciones sobre canales simultáneamente.
<code>defer</code>	Pospone la ejecución de una función hasta el retorno de la función actual.
<code>range</code>	Permite iterar sobre channels hasta su cierre.

`go`

Crea una nueva Goroutine concurrente:

```
go tarea()
```

`defer`

Muy utilizado para liberar recursos:

```
mutex.Lock()  
defer mutex.Unlock()
```

Garantiza liberación incluso ante errores o retornos prematuros.

select

Permite multiplexar múltiples operaciones concurrentes:

```
select {  
  case msg := <-canal1:  
    fmt.Println(msg)  
  
  case <-time.After(time.Second):  
    fmt.Println("timeout")  
}
```

Su comportamiento es similar al `select()` utilizado en sistemas Unix para multiplexación de I/O.

6. Garbage Collector y Concurrency

La administración de memoria en Go se realiza automáticamente mediante un **Garbage Collector (GC)** integrado al Runtime.

El objetivo del GC es detectar objetos que ya no son accesibles por el programa y liberar automáticamente su memoria, evitando:

- memory leaks,
 - punteros colgantes,
 - y errores manuales de liberación (**free**) comunes en lenguajes como C o C++.
-

¿Por qué el Garbage Collector es importante en concurrency?

En sistemas concurrentes pueden existir miles o millones de Goroutines creando objetos dinámicamente al mismo tiempo:

```
go procesarConexion()
```

Cada Goroutine puede:

- reservar memoria,
- crear slices,
- maps,
- channels,
- estructuras dinámicas,
- buffers de red.

Sin un recolector eficiente, la memoria crecería indefinidamente.

Por eso el GC de Go fue diseñado específicamente para trabajar junto al scheduler concurrente.

Go implementa un GC:

- concurrente,
- tri-color,
- mark-and-sweep,
- de baja latencia.

El recolector trabaja simultáneamente con las Goroutines activas minimizando pausas, lo cual resulta fundamental en servidores y sistemas de alta concurrency.

Interacción entre Goroutines y GC

Cada Goroutine posee su propio stack dinámico.

El GC debe analizar constantemente esos stacks para detectar referencias activas.

Por eso el scheduler y el Garbage Collector están profundamente integrados:

- el scheduler administra ejecución,
- el GC inspecciona stacks y heap,
- ambos cooperan para mantener baja latencia.

Incluso durante una operación de Garbage Collection, las Goroutines pueden seguir ejecutándose parcialmente.

7. Conclusión

La concurrencia en Go no se limita únicamente al uso de Goroutines, sino que depende de una arquitectura compleja implementada dentro del Runtime.

El modelo GMP, junto con mecanismos como:

- Work Stealing,
- Local Run Queues,
- Handoff,
- Network Polling,
- y Garbage Collection concurrente,

permiten que Go gestione millones de tareas concurrentes de manera eficiente y escalable.

Gracias a esta abstracción, el desarrollador puede enfocarse en la lógica del problema sin administrar manualmente threads, locks complejos o planificación del CPU, manteniendo al mismo tiempo un alto nivel de rendimiento.